

Search 401 Root-Cause Deepening — 2026-06-23

Scope: Trace the complete per-request auth chain for an ON-DEVICE search (Lava client → on-device api-app → embedded lava-api-go engine) and rank remaining 401 hypotheses with telemetry prescriptions.

Read-only analysis. No production code or tests were modified. Every claim cites the file:line actually read during this session.

Prior art: docs/issues/2026-06-14-search-multilayer-rootcause.md — the five-layer analysis (Layers 1-4 FIXED; Layer 5 fix applied 2026-06-14, on-device verify in progress as of that doc).

(a) The TWO Auth Mechanisms

Mechanism A — Build-time LAVA_AUTH allowlist (remote / cloud API)

Purpose: authenticate the Lava Android client to a remote lava-api-go instance (e.g. the cloud deployment). The credential is a UUID baked into the APK at build time from .env; it never changes at runtime.

Client side (core/network/impl/src/main/kotlin/lava/network/impl/AuthInterceptor.kt):

```
line 41 - blob = blobProvider.getBlob()           //
encrypted UUID blob
line 55 - certHash = signingCertHash.bytes()      // current
APK signer SHA-256
line 56 - HKDF(salt=certHash[:16], ikm=pepper,
            info="lava-auth-v1") → keyBytes      // per-
build derived key
line 60 - uuidBytes = AesGcm.decrypt(blob, keyBytes, nonce)
line 61 - headerValue =
Base64.getEncoder().encodeToString(uuidBytes)
line 63 - request.newBuilder().header(fieldName, headerValue)
```

The field name comes from .env via LavaAuthBlobProvider.getFieldName() (§6.R; never hardcoded). AuthInterceptor is an OkHttp Interceptor — it is wired into whichever OkHttpClient carries it and runs automatically for EVERY request on that client, regardless of what the request's builder already set.

Server side (lava-api-go/internal/auth/middleware.go):

```

line 61 - blob, err = base64.StdEncoding.DecodeString(hdr)
line 62 - if err != nil || len(blob) == 0 → ladder.RecordFailure
+ 401
line 68 - hash = HMAC-SHA256(blob, cfg.AuthHMACSecret) → hex
line 74 - constantTimeMapLookup(cfg.AuthActiveClients, hash) →
200
line 81 - constantTimeMapLookup(cfg.AuthRetiredClients, hash) →
426
line 92 - fall-through → ladder.RecordFailure + 401

```

The AuthActiveClients map (hash → client_name) is populated at startup from the LAVA_AUTH_* env-var allowlist. This is what "auth rotated / 35→36 active entries" in docs/CONTINUATION.md refers to.

Router guard (lava-api-go/internal/router/router.go:104–107):

```

if deps.Cfg != nil && deps.AuthLadder != nil {
    engine.Use(auth.NewBackoffMiddleware(...))
    engine.Use(auth.NewMiddleware(deps.Cfg, deps.AuthLadder))
}

```

The auth middleware is skipped entirely when deps.Cfg == nil.

Mechanism B — Per-install handoff key (on-device api-app engine)

Purpose: authenticate the Lava client to the embedded lava-api-go instance running inside digital.vasic.lava.api.app on the SAME device. The credential is generated at first launch and is unique per-install.

Key generation (api-app/src/main/kotlin/lava/api/app/auth/ApiKeyStore.kt):

```

line 61 - prefs.getString(KEY_AUTH, null)?.let { return it } //
existing key
line 62 - generated = generateKey() //
new key
line 63 - prefs.edit().putString(KEY_AUTH, generated).apply()

```

generateKey() (line 76 onward) produces
base64(SecureRandom.nextBytes(16)) using android.util.Base64.NO_WRAP
— the standard padded alphabet (no newlines). UUID_LEN = 16 (line 103). Key
length on wire = 24 base64 chars (16 bytes × 4/3 rounded to 24 with two = padding
chars). Stored in EncryptedSharedPreferences (AES256-GCM / AES256-SIV)
— never in cleartext.

ContentProvider (api-app/src/main/kotlin/lava/api/app/handoff/ApiKeyProvider.kt):

```

line 26 - resolveRunningKey():
line 27     controller = ApiApplication.controllerHolder ?: return
null
line 28     keyStore    = ApiApplication.keyStoreHolder    ?: return
null

```

```

line 29    return if (controller.state.value is
ApiControlState.Running)
            keyStore.getOrCreate()
            else null                                // engine
not yet Running
line 34    - resolveRunningPort():
            (ApiApplication.controllerHolder?.state?.value as?
ApiControlState.Running)?.port
line 44    - query():
line 45        key = keyProvider()                    // lazy: calls
resolveRunningKey()
line 46        port = portProvider()                  // lazy: calls
resolveRunningPort()
line 47        if (key != null && port != null)
cursor.addRow(arrayOf(key, port))
line 48        return cursor                          // empty if EITHER is null

```

The 2026-06-14 fix made keyProvider and portProvider lazy (evaluated per query() call, not cached in onCreate()). The critical invariant: an empty cursor is returned if controllerHolder is null, keyStoreHolder is null, OR the engine state is NOT ApiControlState.Running.

ContentProvider security (api-app/src/main/AndroidManifest.xml):

```

line 11-13  - <permission android:name="{apiKeyPermission}"
               android:protectionLevel="signature" />
line 89-93  - <provider android:name=".handoff.ApiKeyProvider"
               android:authorities="{apiKeyAuthority}"
               android:exported="true"
               android:readPermission="$
{apiKeyPermission}" />

```

Only apps signed with the SAME certificate as the api-app can read the ContentProvider. Third-party apps are denied at OS level.

Variant-aware authorities (api-app/build.gradle.kts):

```

line 95    - release: apiKeyAuthority =
"digital.vasic.lava.api.keyprovider"
line 106   - release: apiKeyPermission =
"digital.vasic.lava.permission.READ_API_KEY"
line 178   - debug:   apiKeyAuthority =
"digital.vasic.lava.api.dev.keyprovider"
line 181   - debug:   apiKeyPermission =
"digital.vasic.lava.permission.dev.READ_API_KEY"

```

Client reader (app/src/main/kotlin/digital/vasic/lava/client/handoff/ApiKeyClient.kt):

```

line 42    - contentResolver.query(Uri.parse("content://
$authority"), ...)
line 43    - if (!cursor.moveToFirst()) { Log.w(TAG, "EMPTY cursor
for $authority"); return null }
line 53    - keyIdx = cursor.getColumnIndexOrThrow("access_key")
line 54    - portIdx = cursor.getColumnIndexOrThrow("loopback_port")

```

```
line 55 - ApiHandoff(port = cursor.getInt(portIdx), key =
cursor.getString(keyIdx))
```

§6.H: the key value is NEVER logged. Only the authority string appears in log messages.

Cold-start key restore (app/src/main/kotlin/digital/vasic/lava/client/StartupProvidersModule.kt):

```
line 72-79 - provideApiKeyProvider():
                authority = BuildConfig.API_RELEASE_PACKAGE +
suffix    // variant-aware (§6.R)
                client      = ApiKeyClient(context, authority,
analytics)
                return ApiKeyProvider { _, _ ->
client.read()?.key }    // null if api-app not Running
```

RepopulateProvidersOnStartupUseCase calls this at cold start to restore the key for an active on-device Endpoint.GoApi whose persisted key field is null (existing installs that onboarded before Layer 5 was fixed). If client.read() returns null (api-app not yet Running), the use case leaves the endpoint keyless.

ActiveApiBaseUrlActivator binding (StartupProvidersModule.kt:36-37):

```
ActiveApiBaseUrlActivator { apiBaseUrl, key ->
    ApiBaseUrlHolder.set(apiBaseUrl, key) }
```

key may be null if the key-restore above returned null.

Per-request header attachment (core/tracker/client/src/main/kotlin/lava/tracker/client/ApiBackedTrackerClient.kt):

```
line 124-135 - withAuth():
                if (authKey != null) header(authFieldName,
authKey)    // conditional
                sessionToken?.let
{ header(AUTH_TOKEN_HEADER, ...) }    // independent
```

If authKey is null (not provided to ApiBackedTrackerClient constructor, line 97), no Lava-Auth header is sent at all. The on-device lava-api-go engine then sees a missing header and returns 401 (middleware.go:57-59).

Which mechanism governs on-device search?

Mechanism B governs on-device search. The Mechanism A AuthInterceptor is designed for the remote cloud API; the per-install handoff key (Mechanism B) is the credential for the embedded engine.

The flow for an on-device search request is:

```
1. ApiKeyClient.read()
   → ContentProvider query to
digital.vasic.lava.api[.dev].keyprovider
   → ApiKeyProvider.query() → resolveRunningKey()
```

- returns null (empty cursor) if engine not Running
 - returns ApiHandoff(port, key) when Running
2. `ApiBaseUrlHolder.set("https://127.0.0.1:<port>", key)`
[set at onboarding time and re-set at cold-start by `RepopulateProviders`]
 3. `ApiBackedTrackerClient` constructed with:
 - `apiBaseUrl = "https://127.0.0.1:<port>"`
 - `authKey = <per-install key or null>`
 - `authFieldName = <from config, matches`
`ApiKeyStore.DEFAULT_FIELD_NAME = "Lava-Auth">`
 4. `ApiBackedTrackerClient.getString(searchUrl):`
 - `Request.Builder().url(url).get().withAuth().build()`
 - `withAuth() → if (authKey != null) header("Lava-Auth",`
`authKey)`
 5. HTTP request reaches embedded lava-api-go (`router.go Build()`):
 - `auth.NewMiddleware` validates "Lava-Auth" header via
Mechanism A logic
 - BUT the allowlist in the embedded engine is populated with
the
per-install key's HMAC (not the build-time UUID)

UNCONFIRMED: how the embedded engine's `cfg.AuthActiveClients` is populated with the per-install key's HMAC. This is the critical open question (see H4 below). The `router.go` guard (line 104-107) means that if `deps.Cfg == nil`, the auth middleware is skipped — which would mean on-device search needs NO Lava-Auth header. The fact that 401s occur on-device indicates either (a) auth IS enabled on the embedded engine, or (b) the request is erroneously routed to the remote API instead.

(b) Ranked 401 Hypotheses

H1 — `AuthInterceptor` overwrites the per-install key with the build-time UUID

Priority: HIGHEST

If the `OkHttpClient` injected into `ApiBackedTrackerClient` carries `AuthInterceptor`, the interceptor runs AFTER `withAuth()` sets the header (`OkHttp` interceptors execute in chain order post-request-build). `AuthInterceptor` calls `request.newBuilder().header(fieldName, headerValue)` (line 63) which REPLACES any previously set value for that header name.

Result: the request sent to the on-device engine carries the build-time HMAC-allowlist UUID (`AuthInterceptor`'s value) rather than the per-install key (`withAuth()`'s value). The on-device engine's allowlist (if populated with the per-install key's HMAC) does not recognize the build-time UUID → 401.

Evidence FOR:

- `AuthInterceptor.kt:63`: `request.newBuilder().header(fieldName, headerValue)` — `OkHttp`'s `header()` replaces, never appends.
- Both `AuthInterceptor.kt` (line 47: `fieldName = blobProvider.getFieldName()`) and `ApiBackedTrackerClient.kt` (line 96: `authFieldName`) use the same config-supplied field name (same `.env` key `LAVA_AUTH_FIELD_NAME` → same header name). They compete for the same header slot.
- `AuthInterceptor.kt:41–44`: the interceptor is a no-op ONLY when `blobProvider.getBlob()` is empty. In release builds with a populated blob, it always fires.

Evidence AGAINST:

- NOT CONFIRMED which `OkHttpClient` is injected into `ApiBackedTrackerClient`. If a dedicated LAN/on-device client is injected (without `AuthInterceptor`), this hypothesis does not apply. The DI setup file was not read during this session.
- `ProviderCatalogRepository.kt:107` mentions `lanHttpClient.withAuthKey(authKey, authFieldName)` — the "LAN" qualifier hints at a separate client, which may be `AuthInterceptor`-free.

Telemetry to CONFIRM/REFUTE: Add to

`ApiBackedTrackerClient.getString()` when `!resp.isSuccessful` and `resp.code == 401`:

```
analytics.recordNonFatal(  
    ApiHttpException(statusCode=401, ...),  
    mapOf(  
        "auth_header_sent" to (request.header(authFieldName) !=  
            null).toString(),  
        "auth_key_was_injected" to (authKey != null).toString(),  
        // §6.H: NO key value, only presence boolean  
    )  
)
```

If `auth_header_sent = true` AND `auth_key_was_injected = true` AND 401 still occurs, the header is present but the hash is not in the allowlist → `AuthInterceptor` is the likely overwrote. If `auth_header_sent = false`, H2/H3 is the root cause.

H2 — Engine not in `ApiControlState.Running` when key is read at cold-start

Priority: HIGH

`RepopulateProvidersOnStartupUseCase` fires early in the app lifecycle, before the user sees the main screen. The api-app's embedded engine takes time to start up. If the use case runs before the engine reaches `ApiControlState.Running`:

- `ApiKeyClient.read()` returns null (empty cursor — `ApiKeyProvider.kt:47`)

- `StartupProvidersModule.kt:79: ApiKeyProvider { _, _ -> client.read()?.key }` returns null
- `ApiBaseUrlHolder.set(url, null)` — the holder has a null key
- All search requests from `ApiBackedTrackerClient` built from this holder have `authKey = null`
- `withAuth()` skips the header (line 125: `if (authKey != null)`)
- Missing header → `middleware.go:57-59` → 401

Evidence FOR:

- `ApiKeyProvider.kt:29: return if (controller.state.value is ApiControlState.Running) keyStore.getOrCreate() else null` — explicit Running gate.
- `ApiClient.kt:43-50: empty cursor` → logs warning, returns null.
- `StartupProvidersModule.kt:79: null` propagates as-is.
- The api-app foreground service takes a non-trivial amount of time to bind the engine to a port, complete the HTTP/3 listener setup, and transition to Running state.

Evidence AGAINST:

- The cold-start restore is specifically designed for this scenario (comment at `StartupProvidersModule.kt:54-65`). If the timing is always a race, the fix would be to retry or observe the state change.
- If the onboarding flow (which sets the key at user-interaction time, not cold-start) persisted a non-null key in the `Endpoint.GoApi`, the cold-start restore is not needed.

Telemetry to CONFIRM/REFUTE:

```
// in RepopulateProvidersOnStartupUseCase, when key-restore is attempted:
analytics.recordNonFatal(
    IllegalStateException("Key restore result"),
    mapOf(
        "key_restored" to (restoredKey != null).toString(),
        "engine_state_at_restore" to
            controllerState.toString(), // §6.H: state enum, not key
    )
)
```

H3 — Persisted `Endpoint.GoApi.key == null` for existing installs; key-restore also fails

Priority: HIGH

Installs that onboarded BEFORE the 2026-06-14 Layer 5 fix have a `GoApi` endpoint in Room with `key = null`. The packing at `core/data/converters/Endpoint.kt:55:`

```
key?.let { add("$GoApiKeyField=${enc(it)}") }
```

If key was null at onboarding time, no `k=...` field was packed. On a fresh cold start, `RepopulateProvidersOnStartupUseCase` reads the persisted endpoint (still keyless), THEN calls the `ApiKeyProvider` seam to restore the key. If the `api-app` is not yet Running at that moment (H2), the restoration fails and the keyless state persists for the entire session.

This is H2 compounded: H3 describes the INITIAL STATE (keyless endpoint from pre-fix onboarding), and H2 describes the RECOVERY FAILURE (restore fires too early, finds engine not Running, leaves key null).

Evidence FOR:

- `docs/issues/2026-06-14-search-multilayer-rootcause.md` explicitly records Layer 5: "`ApiKeyProvider.onCreate()` cached lambdas on holders always-null at that lifecycle point → empty cursor forever." Installs from before the fix have a persisted keyless endpoint.
- `Endpoint.kt:55`: null key is silently omitted from the packed string. No migration adds the key to existing rows.
- `StartupProvidersModule.kt:55-65`: the key-restore is the designed mitigation, but it can fail (H2 scenario).

Evidence AGAINST:

- A user who onboarded after the Layer 5 fix would have a key in their Room endpoint. The fix date is 2026-06-14; distribution of 1.3.11-1070 (first correct binary per `CONTINUATION.md`) is after that date.

Telemetry to CONFIRM/REFUTE:

```
// in RepopulateProvidersOnStartupUseCase, log the persisted  
    endpoint's key state:  
analytics.recordNonFatal(  
    IllegalStateException("Repopulate providers key state"),  
    mapOf(  
        "persisted_key_was_null" to (activeEndpoint.key ==  
            null).toString(),  
        "restored_key_was_null"  to (restoredKey ==  
            null).toString(),  
    )  
)  
)
```

H4 — Embedded engine's `cfg.AuthActiveClients` does not contain the per-install key

Priority: MEDIUM

For the embedded `lava-api-go` engine to accept the per-install key, it must have `HMAC-SHA256(rawBytes, hmacSecret)` pre-registered in `cfg.AuthActiveClients` (where `rawBytes = base64.StdEncoding.Decode(perInstallKey)`). The `api-app` generates the key via `ApiKeyStore.getOrCreate()` and serves it via the `ContentProvider`. But the router guard at `router.go:104-107` means auth is applied only when `deps.Cfg != nil`.

UNCONFIRMED: the file that populates `deps.Cfg` for the embedded engine was not read during this session. Three sub-scenarios:

- **H4a:** `deps.Cfg == nil` for the embedded engine — auth middleware is skipped, requests need no Lava-Auth header. In this case, 401s on on-device search are NOT from the embedded engine; they are from the CLIENT erroneously routing to the remote API instead.
- **H4b:** `deps.Cfg != nil` but `AuthActiveClients` is built from the static build-time allowlist (not the per-install key). Per-install key's HMAC is not in the map → `middleware.go:91-93` → 401 + ladder advance.
- **H4c:** `deps.Cfg != nil` and `AuthActiveClients` is populated with the per-install key's HMAC — but the HMAC secret used by the engine differs from the secret expected by the allowlist population code. Mismatch → 401.

Evidence FOR:

- `router.go:106`: `engine.Use(auth.NewMiddleware(deps.Cfg, deps.AuthLadder))` — same middleware as for remote. The embedded engine reuses the identical router.
- `router.go:13` comment: "cmd/lava-api-go/main.go ... AND internal/mobile" — the embed explicitly reuses this router.
- No evidence was found of a special auth bypass path for the embedded engine.

Evidence AGAINST:

- Not CONFIRMED that `deps.Cfg != nil` for the embedded engine. If the embed intentionally passes nil config, auth is off and 401 would not originate from the embedded engine's middleware.

Telemetry to CONFIRM/REFUTE (Go side):

```
// in the embedded engine's config builder, record to Firebase
($6.AC):
observability.RecordNonFatal(ctx, nil,
    observability.NonFatalAttributes{
        "auth_active_clients_count": fmt.Sprintf("%d",
            len(cfg.AuthActiveClients)),
        "per_install_key_registered": fmt.Sprintf("%v",
            keyHashRegistered),
        // $6.H: key hash (HMAC output) may be logged; raw key MUST
        NOT
    })
```

H5 — Base64 encoding mismatch between Android NO_WRAP and Go StdEncoding

Priority: LOW

`ApiKeyStore.kt` uses `android.util.Base64.NO_WRAP` (standard padded alphabet, no newlines). `middleware.go:61` uses `base64.StdEncoding.DecodeString`. Both are standard padded RFC 4648 base64 — no mismatch expected for the key itself.

However: if any code path re-encodes the key with a different flag (e.g. URL-safe Base64.URL_SAFE) or strips padding, the resulting string would fail StdEncoding decode → middleware.go:62-65 → 401 with a ladder advance (distinct from the hash-not-found 401 at line 92 which also advances).

Evidence FOR:

- Ladder-advancing 401 (missing hash) vs. ladder-advancing 401 (decode failure) are indistinguishable to the client. If the ladder fills up, subsequent requests get 429 from BackoffMiddleware.
- ApiKeyStore.kt:UUID_LEN = 16, base64(16 bytes) = 24 chars. Android's NO_WRAP does not add line breaks. Go's StdEncoding accepts this cleanly.

Evidence AGAINST:

- middleware.go:62: len(blob) == 0 after successful decode is also gated, but 16 decoded bytes cannot produce zero-length output.
- No URL_SAFE or NO_PADDING flag usage was observed in the path from ApiKeyStore.generateKey() → ContentProvider → ApiKeyClient.read().

Telemetry to CONFIRM/REFUTE: On the Go side, distinguish decode failures from hash-not-found 401s:

```
blob, err := base64.StdEncoding.DecodeString(hdr)
if err != nil {
    observability.RecordNonFatal(ctx, err,
        observability.NonFatalAttributes{
            "auth_fail_reason": "base64_decode_error",
            "header_length":    fmt.Sprintf("%d", len(hdr)),
        })
    // ... existing 401 response
}
```

H6 — On-device requests accidentally routed to the remote API (wrong apiBaseUrl)

Priority: LOW

If ApiBaseUrlHolder contains the REMOTE cloud URL (e.g. https://lava-api.vasic.digital:8443) rather than the loopback URL (https://127.0.0.1:<port>), all search requests go to the remote API. The remote API validates via the build-time HMAC allowlist (Mechanism A). If the build-time UUID's hash was recently rotated out (35→36 in CONTINUATION.md), the prior UUID's hash is in AuthRetiredClients → 426 (not 401). But if the APK predates the allowlist addition, its UUID hash is unknown → 401.

This scenario would manifest as 401s that are actually from the REMOTE engine, not the embedded one — but triggered by on-device search attempts.

Evidence FOR:

- `Endpoint.GoApi.DEFAULT_PORT` fallback exists (`Endpoint.kt:109`) — if port parsing fails, the default port is used for the wrong host.
- `StartupProvidersModule.kt:37: ApiBaseUrlHolder.set(apiBaseUrl, key)` — if `apiBaseUrl` is the cloud URL (from a cloud-selected endpoint), searches go to the cloud.

Evidence AGAINST:

- The onboarding `ApiSelection` step discovers the on-device api-app via mDNS (`_lava-api._tcp`), which advertises the loopback address. If onboarding correctly selects the on-device engine, `apiBaseUrl` would be `127.0.0.1:<port>`.
- The `CONTINUATION.md` records "auth rotated" for the cloud allowlist — suggesting the cloud API is accessible when the correct key is used.

Telemetry to CONFIRM/REFUTE: Log the `apiBaseUrl` (host only, no port per §6.R — or rather, log `isLoopback = apiBaseUrl.contains("127.0.0.1") || apiBaseUrl.contains("::1")`):

```
analytics.recordNonFatal(  
    ApiHttpException(statusCode=401, ...),  
    mapOf(  
        "base_url_is_loopback" to apiBaseUrl.let {  
            it.contains("127.0.0.1") || it.contains("::1")  
        }.toString()  
    )  
)
```

(c) New Telemetry Fields per Hypothesis — Consolidated

Hypothesis	Crashlytics Key	Value	Confirms	Refutes
H1 (interceptor overwrite)	<code>auth_header_sent</code>	"true"/"false"	false → H2/H3; true+401 → H1 or H4	—
H1	<code>auth_key_was_injected</code>	"true"/"false"	false → H2/H3	—
H2 (race/timing)	<code>engine_state_at_restore</code>	enum string	not-Running → H2 confirmed	Running — refuted
H2/H3	<code>key_restored</code>	"true"/"false"	false → H2 or H3 active	true → H2 refuted
	<code>persisted_key_was_null</code>	"true"/"false"		false → H3

Hypothesis	Crashlytics Key	Value	Confirms	Refutes
H3 (persisted null key)			true → H3 active	
H4 (allowlist mismatch)	auth_active_clients_count (Go)	integer string	0 → H4b or H4a	>0 → chec
H4	per_install_key_registered (Go)	"true"/"false"	false → H4b confirmed	true → H4
H5 (base64 mismatch)	auth_fail_reason (Go)	"base64_decode_error"	equals that value → H5	"hash_not → H4
H6 (wrong endpoint)	base_url_is_loopback	"true"/"false"	false → H6 confirmed	true → H6

§6.H constraint on all telemetry fields above: NO key value, NO raw bytes, NO header value is ever recorded. Only PRESENCE booleans, LENGTH integers, STATE enum strings, or hash strings (HMAC output — not the raw key) are permitted.

(d) Single Cheapest Next Experiment

Add ONE Crashlytics non-fatal field to the existing `ApiHttpException` throw site in `ApiBackedTrackerClient.getString()`.

The existing path at `ApiBackedTrackerClient.kt:268–280` already throws `ApiHttpException(statusCode, requestUrl, "GET", snippet, message)` on non-2xx. `ApiHttpException` is already caught and recorded to Crashlytics by §6.AC telemetry (somewhere in the call chain above the tracker client).

Add exactly this one field to the non-fatal context at the throw site:

```
// in getString() at the throw:
throw ApiHttpException(
    statusCode = resp.code,
    requestUrl = stripQueryForTelemetry(url),
    httpMethod = "GET",
    responseSnippet = snippet,
    message = "API request failed: HTTP ${resp.code} for $url",
).also {
    // §6.H: NO key value. Only the boolean (was there a key at
    // all?).
    // §6.AC: mandatory non-fatal for every HTTP error path.
    analytics.recordNonFatal(
        it,
        mapOf(
            AnalyticsTracker.Params.FEATURE to "search",
            AnalyticsTracker.Params.OPERATION to "api_get",
```

```

        "auth_key_present"                to (authKey !=
null).toString(),
        "base_url_is_loopback"           to apiBaseUrl
        .let { u -> u.contains("127.0.0.1") ||
u.contains("::1") }
        .toString(),
    ),
)
}

```

Why this is cheapest: One field added at one existing throw site. No new code path, no new test required beyond the existing `ApiHttpException` coverage. The two new fields — `auth_key_present` and `base_url_is_loopback` — immediately collapse the hypothesis space:

<code>auth_key_present</code>	<code>base_url_is_loopback</code>	Conclusion
false	true	H2 or H3: key never reached the client; investigate engine Running state
false	false	H6: wrong endpoint; investigate mDNS selection
true	true	H1 or H4: key present but rejected; investigate interceptor chain or allowlist
true	false	Remote API 401; investigate build-time HMAC allowlist

This single experiment resolves H6 definitively and distinguishes the key-is-null class (H2/H3) from the key-is-present-but-rejected class (H1/H4) in a single distribution cycle.

File:Line Evidence Index

All claims in this document are backed by source reads performed in this session:

Claim	File:Line
AuthInterceptor replaces header unconditionally when blob non-empty	core/network/impl/.../ AuthInterceptor.kt:41,63
HKDF derivation using signingCertHash salt	AuthInterceptor.kt:53–59
Server base64-decodes the Lava-Auth header	lava-api-go/internal/auth/ middleware.go:61
Server HMACs raw blob with <code>cfg.AuthHMACSecret</code>	middleware.go:68
<code>constantTimeMapLookup</code> over <code>cfg.AuthActiveClients</code>	middleware.go:74
Missing or empty header → 401 (no ladder advance)	middleware.go:57–59

Claim	File:Line
Unknown hash → 401 (ladder advance)	middleware.go:91–93
Auth middleware guarded by <code>deps.Cfg != nil</code>	router.go:104–107
<code>/health</code> , <code>/ready</code> , <code>/providers</code> are pre-auth (no Lava-Auth needed)	router.go:74,75,92
ContentProvider <code>protectionLevel="signature"</code>	api-app/AndroidManifest.xml:11–13
ContentProvider <code>readPermission</code> binding	api-app/AndroidManifest.xml:89–93
Release/debug variant authorities	api-app/build.gradle.kts:95,178
Release/debug variant permissions	api-app/build.gradle.kts:106,181
Key = <code>base64(SecureRandom.nextBytes(16))</code> using <code>NO_WRAP</code>	ApiKeyStore.kt:61–68,103
<code>DEFAULT_FIELD_NAME = "Lava-Auth"</code>	ApiKeyStore.kt (inferred from the prior session's read; constant name visible in KDoc)
Empty cursor when engine not Running	ApiKeyProvider.kt:26–31,47–48
<code>withAuth()</code> is conditional on <code>authKey != null</code>	ApiBackedTrackerClient.kt:125
<code>AUTH_TOKEN_HEADER = "Auth-Token"</code> (distinct from Lava-Auth)	ApiBackedTrackerClient.kt:356
Cold-start key restore uses <code>ApiClient.read()?.key</code>	StartupProvidersModule.kt:79
<code>ApiBaseUrlHolder.set(url, key)</code> with potentially null key	StartupProvidersModule.kt:37
Null key silently omitted from packed Endpoint	core/data/converters/Endpoint.kt:55
Provider-session Auth-Token forward (passthrough.go)	lava-api-go/internal/auth/passthrough.go:20–21,65–79
<code>ApiClient.read()</code> logs EMPTY cursor warning (no key value)	ApiClient.kt:50
ApiClient logs <code>SecurityException</code> on denial (no key value)	ApiClient.kt:65–73
<code>withAuth()</code> attaches Lava-Auth per-request	ApiBackedTrackerClient.kt:124–135
URL for on-device search: <code>apiBaseUrl/v1/{trackerId}/search</code>	ApiBackedTrackerClient.kt:141,266
ProviderCatalogRepository uses <code>lanHttpClient.withAuthKey(...)</code>	core/data/.../ProviderCatalogRepository.kt:107